Neural Networks & Deep Learning

Revision cards: the key concepts of the course

MSA-DATB09-03, sessions 2 to 8

How each card works

IN ONE LINE: what it is
EXAMPLE: the idea in everyday words, then a diagram
THE MATHS: the formulas, as in the course
WORKED EXAMPLE: small numbers, all recomputed
IN PYTORCH: the line of code
IN OUR PROJECT: where we met it in the museum project
REMEMBER: the trap or the sentence to keep

Contents

Session 2 · The sigmoid neuron 3
1. The sigmoid neuron 3
2. The log loss (binary cross-entropy) 4
3. The gradient (a − y) x 5
4. From NumPy to PyTorch 6
Session 3 · A network and backpropagation 7
5. A neural network (MLP) 7
6. Backpropagation by hand 8
7. Autograd 9
Session 4 · PyTorch in depth 10
8. Tensors 10
9. Modules (nn.Module, nn.Linear, nn.Sequential) 11
10. Losses: softmax and cross-entropy 12
11. Optimisers and the training loop 13
Session 5 · Activations and the vanishing gradient 14
12. ReLU 14
13. The vanishing gradient 15
Session 6 · Better training 16
14. Weight decay (L2 regularisation) 16
15. Dropout 17
16. Early stopping 18
17. Batch normalisation 19
18. Momentum and Adam 20
19. Diagnosing a training run 21
Session 7 · Convolutional networks 22
20. Convolution 22
21. Pooling 23
22. ResNet 24
23. Transfer learning 25
Session 8 · Sequences and attention 26
24. Recurrent neural network (RNN) 26
25. LSTM 27
26. Attention 28
27. Embeddings 29
One-line summary 30

Session 2 · The sigmoid neuron

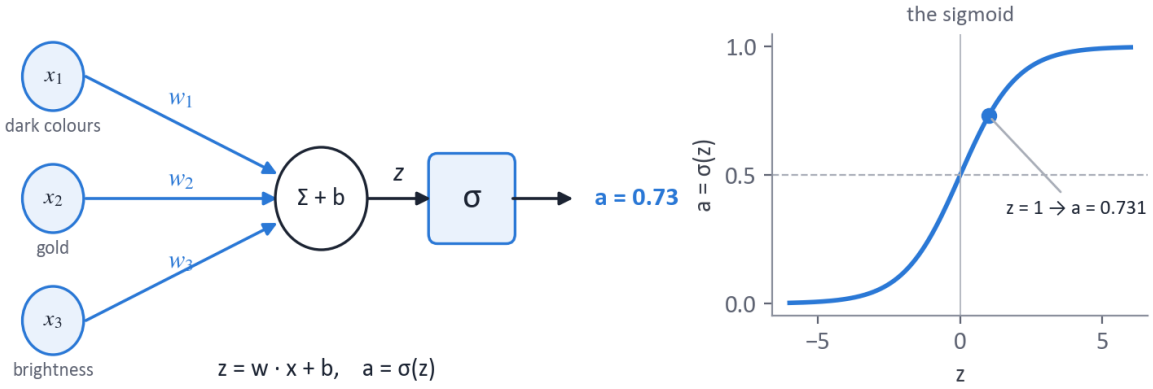
What we built: Sigmoid neuron: log loss, gradient (a − y)x, NumPy then PyTorch

Its limit, fixed next: still one straight line

1. The sigmoid neuron

IN ONE LINE **A neuron computes a weighted sum of its inputs, then squeezes it into a probability between 0 and 1.**

EXAMPLE Is this painting Baroque? The neuron weighs the clues (dark colours push up, bright colours push down), adds them, and the sigmoid turns the total into a probability, for example 0.73.



THE MATHS

$z = w \cdot x + b$ (weighted sum)
$a = \sigma(z) = 1 / (1 + e^{(-z)})$
$\sigma(0) = 0.5,\ \sigma(\text{large } z) \to 1,\ \sigma(\text{very negative } z) \to 0$
decide class 1 if $a > 0.5$, that is if $w \cdot x + b > 0$: a straight line

WORKED EXAMPLE

$x = 2, w = 0.5, b = 0 \ \to\ z = 1 \ \to\ a = \sigma(1) = 0.731$

IN PYTORCH

```
a = torch.sigmoid(w * x + b)
```

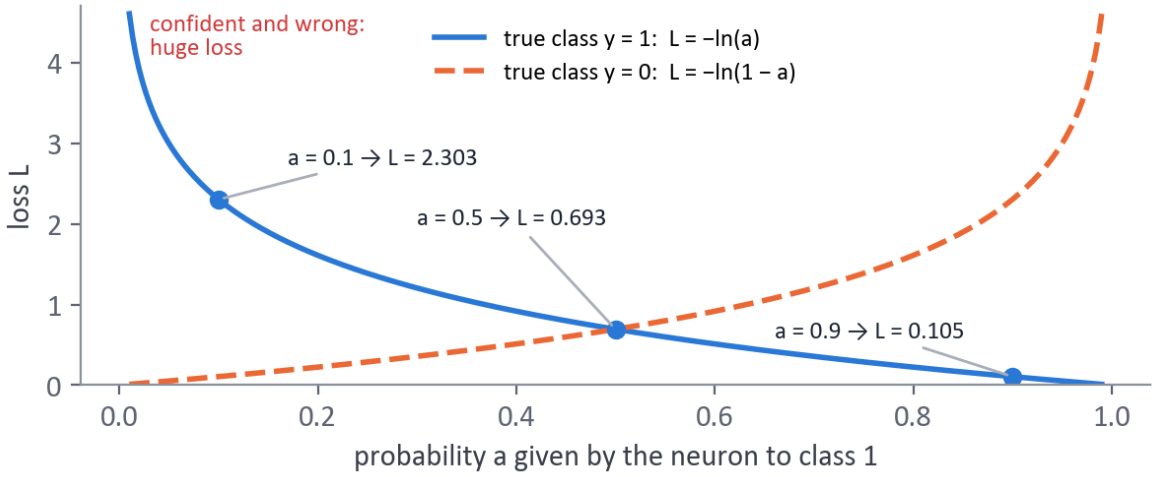
IN OUR PROJECT Our broken run with 8 sigmoid layers could not learn (see the vanishing gradient). For our 5 styles we use the softmax, the multi-class cousin of the sigmoid.

REMEMBER **One neuron draws one straight boundary. To draw curves, we need hidden layers (session 3).**

2. The log loss (binary cross-entropy)

IN ONE LINE **The error of a probability: small when the neuron is confident and right, huge when it is confident and wrong.**

EXAMPLE A forecaster who said "90 % sun" on a rainy day should be punished much more than one who said "50 %".



THE MATHS

$L = -[\, y \cdot \ln(a) + (1-y) \cdot \ln(1-a) \,]$
only one term is active: $y = 1 \to L = -\ln(a)$; $y = 0 \to L = -\ln(1-a)$
cost J = mean of L over the m examples (it comes from maximising the likelihood)
why not the squared error? with a sigmoid its gradient has a factor $a(1-a) \approx 0$ when very wrong

WORKED EXAMPLE

$y = 1$: $a = 0.9 \to L = 0.105$; $a = 0.5 \to L = 0.693$; $a = 0.1 \to L = 2.303$
$y = 1, a = 0.01$: squared-error gradient factor -0.0196, log loss -0.99: 50 times more correction

IN PYTORCH

```
loss_fn = nn.BCEWithLogitsLoss()   # takes the raw z: more stable
```

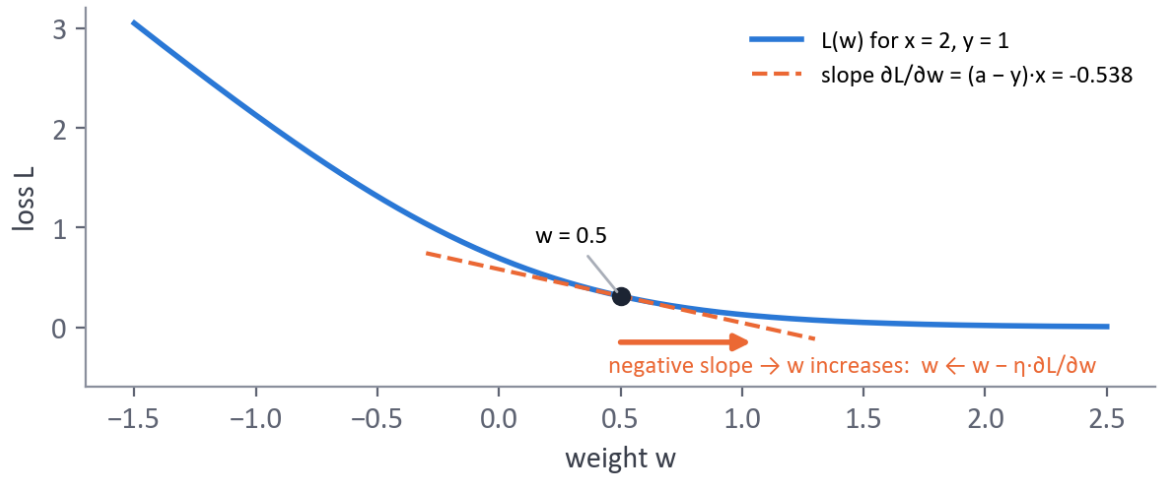
IN OUR PROJECT Our loss for 5 styles is the same idea: L = −ln(probability of the right style).

REMEMBER **Confident and wrong costs a lot. With a sigmoid output, use the log loss, not the squared error.**

3. The gradient (a – y) x

IN ONE LINE **How much, and in which direction, to change each weight to reduce the log loss.**

EXAMPLE The neuron said 0.73 for a true Baroque painting (y = 1): it was 0.27 too low, so the weights of the clues that were present get pushed up.



THE MATHS

chain rule: $\partial L/\partial w = \partial L/\partial a \cdot \partial a/\partial z \cdot \partial z/\partial w$
$\partial L/\partial a = -y/a + (1-y)/(1-a)$, $\partial a/\partial z = a(1-a)$, $\partial z/\partial w = x$
everything simplifies: $\partial L/\partial z = a - y$, so $\partial L/\partial w = (a-y)\cdot x$, $\partial L/\partial b = a - y$
update (gradient descent): $w \leftarrow w - \eta \cdot (a-y) \cdot x$

WORKED EXAMPLE

$a = 0.731, y = 1, x = 2 \rightarrow \partial L/\partial w = (0.731 - 1) \times 2 = -0.538$
with $\eta = 0.1$: $w = 0.5 - 0.1 \times (-0.538) = 0.554$

IN PYTORCH

```
loss.backward()      # w.grad now holds -0.538
```

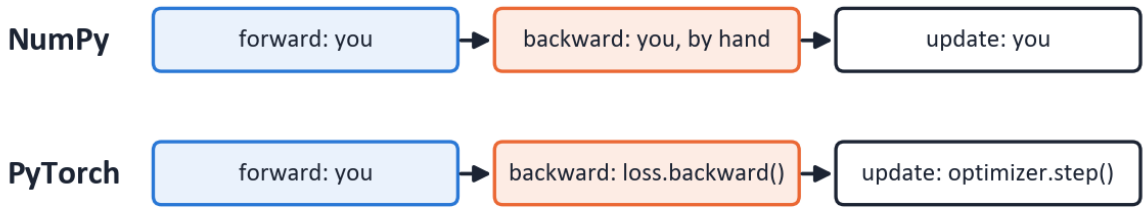
IN OUR PROJECT Softmax + cross-entropy gives exactly the same form: ∂L/∂z = p − y (see the losses card).

REMEMBER **Error × input. A big error or a big input gives a big correction; a correct prediction gives none.**

4. From NumPy to PyTorch

IN ONE LINE **First write the maths by hand with arrays (NumPy), then let PyTorch do the same with tensors and automatic gradients.**

EXAMPLE Like doing long division by hand before using a calculator: you know exactly what the calculator does.



same maths: in PyTorch, autograd computes the gradients for you

THE MATHS

vectorised over m examples (X of shape (n, m)), no Python loop:

$Z = w^T X + b, \quad A = \sigma(Z)$

$dZ = A - Y, \quad dw = (1/m) \cdot X \cdot dZ^T, \quad db = (1/m) \cdot \Sigma\, dZ$

WORKED EXAMPLE

the hand-made gradient and the autograd gradient must be equal: check it once on a small example

IN PYTORCH

```
A = 1 / (1 + np.exp(-(w.T @ X + b)))                    # NumPy
loss = loss_fn(model(X), Y); loss.backward(); optimizer.step()   # PyTorch
```

IN OUR PROJECT The brief asks for networks built from PyTorch primitives: nn.Linear and our own training loop, no magic library.

REMEMBER **PyTorch = NumPy + automatic gradients (+ GPU). Vectorise: one matrix product instead of a loop.**

Session 3 · A network and backpropagation

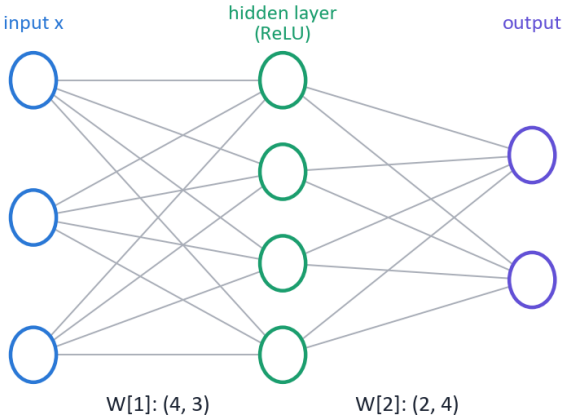
What we built: A network, backpropagation by hand, autograd

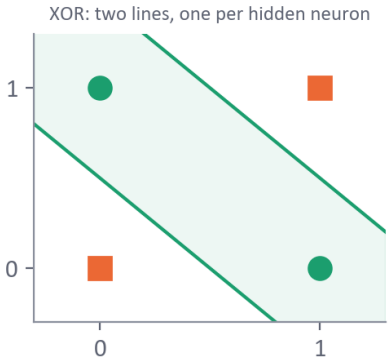
Its limit, fixed next: we need a clean pipeline

5. A neural network (MLP)

IN ONE LINE **Layers of neurons: each layer transforms the output of the previous one, with a non-linear activation in between.**

EXAMPLE XOR (true when exactly one input is true) cannot be separated by one straight line. Two hidden neurons each draw a line, and the output neuron combines them.





THE MATHS

layer l: $z[l] = W[l] \cdot a[l-1] + b[l]$, $a[l] = g(z[l])$, with $a[0] = x$
shapes: $W[l]$ is $(n[l], n[l-1])$, $b[l]$ is $(n[l], 1)$
without g, two layers collapse into one: $W2\ (W1\ x) = (W2\ W1)\ x$
parameters of a dense layer: $n_in \times n_out + n_out$

WORKED EXAMPLE

28×28 image → 784 inputs → 128 hidden → 10 outputs
$W[1]: (128, 784) = 100\,352$ weights + 128 biases; $W[2]: (10, 128) = 1\,280 + 10$; total 101 770

IN PYTORCH

```
nn.Sequential(nn.Flatten(), nn.Linear(784, 128), nn.ReLU(), nn.Linear(128, 10))
```

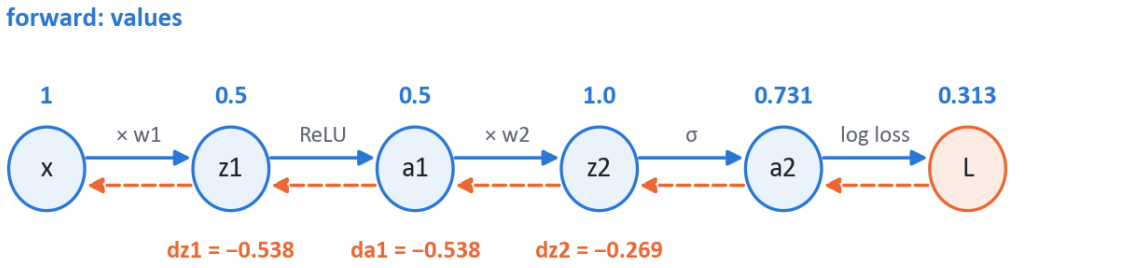
IN OUR PROJECT Our MLP: 12 288 pixels → 512 → 256 → 5, 6.4 million parameters: it learned the paintings by heart (99.7 % train, 40.6 % validation).

REMEMBER **The non-linear activation is what makes depth useful.**

6. Backpropagation by hand

IN ONE LINE **The chain rule applied from the output back to the input, to get the gradient of the loss for every weight in one backward pass.**

EXAMPLE After a wrong answer, the blame travels back through the network: each layer receives its share, in proportion to its contribution.



THE MATHS

output layer (sigmoid + log loss): $dz[L] = a[L] - y$
every layer: $dW[l] = dz[l] \cdot a[l-1]^T$, $db[l] = dz[l]$
one layer back: $da[l-1] = W[l]^T \cdot dz[l]$, $dz[l-1] = da[l-1] \odot g'(z[l-1])$
$\odot$ = element by element

WORKED EXAMPLE

$x = 1 \rightarrow$ hidden ReLU ($w1 = 0.5$) $\rightarrow$ output sigmoid ($w2 = 2$), $y = 1$
forward: $z1 = 0.5, a1 = 0.5, z2 = 1, a2 = 0.731, L = 0.313$
backward: $dz2 = -0.269$, $dw2 = dz2 \cdot a1 = -0.134$, $da1 = w2 \cdot dz2 = -0.538$, $dz1 = -0.538$ (ReLU slope 1), $dw1 = dz1 \cdot x = -0.538$

IN PYTORCH

```
loss.backward()   # gives exactly dw1 = -0.538 and dw2 = -0.134
```

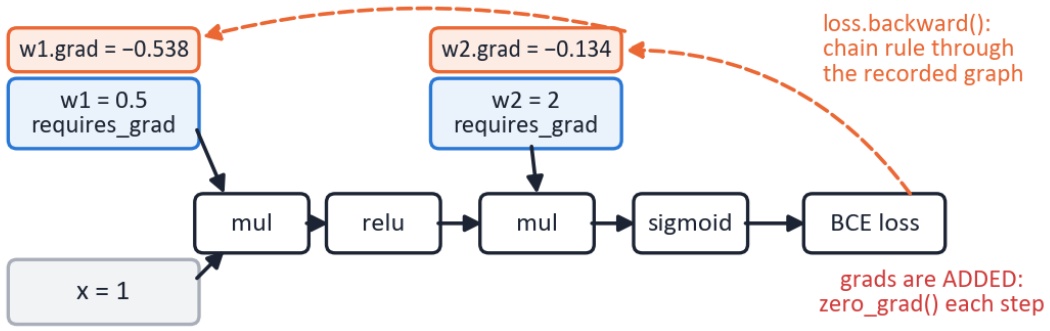
IN OUR PROJECT The gradient per layer is one of our monitoring tools: it revealed the vanishing gradient of the sigmoid network (6×10^{-7}).

REMEMBER **Error of the layer × its input. On the way back, the gradient is multiplied by g' at every layer.**

7. Autograd

IN ONE LINE **PyTorch records every operation on tensors and computes all the gradients automatically with .backward().**

EXAMPLE A cashier who keeps every receipt: at the end of the day, he can trace exactly where each euro came from.



THE MATHS

forward: PyTorch builds a computation graph; each node knows its local derivative
backward: chain rule from the loss to every tensor with requires_grad = True
gradients are ADDED into .grad, hence zero_grad() before each step

WORKED EXAMPLE

on the network of the previous card, autograd gives $dw1 = -0.538$ and $dw2 = -0.134$: the hand values

IN PYTORCH

```
w = torch.tensor(0.5, requires_grad=True)
loss.backward(); print(w.grad)
with torch.no_grad(): ...      # evaluation, no graph
```

IN OUR PROJECT Our broken run "zero_grad forgotten": the gradients piled up, 26 times bigger after 2 epochs. Frozen layers have requires_grad = False: no gradient, no change.

REMEMBER **backward() adds to .grad: always zero_grad(). requires_grad = False freezes a layer.**

Session 4 · PyTorch in depth

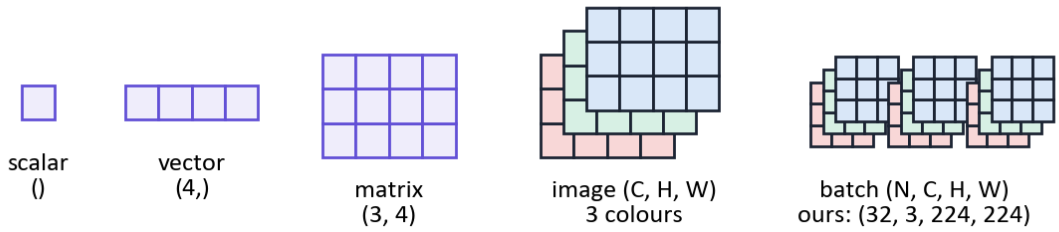
What we built: Tensors, modules, losses, optimisers, the training loop

Its limit, fixed next: deep sigmoid networks do not learn

8. Tensors

IN ONE LINE **Multi-dimensional arrays of numbers, the basic object of PyTorch: inputs, outputs and weights are all tensors.**

EXAMPLE A number, a list, a spreadsheet, a pile of spreadsheets (a colour image), a box of piles (a batch of images).



THE MATHS

shape: scalar (), vector (n,), matrix (m, n), image (C, H, W), batch (N, C, H, W)
dtype: uint8 for raw pixels, float32 for the maths, int64 for class labels
broadcasting: a (1, 3, 1, 1) tensor of means is stretched over a (N, 3, H, W) batch

WORKED EXAMPLE

one batch of our paintings: $(32, 3, 224, 224) = 4\,816\,896$ numbers, 19.3 MB in float32

IN PYTORCH

```
x.shape,  x.float() / 255,  x.flatten(1),  x.to("cuda")
```

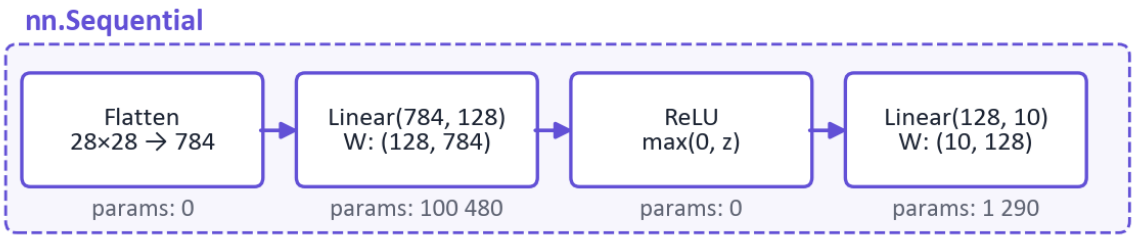
IN OUR PROJECT ResNet features (1 061, 512); logits (32, 5); labels (32,) in int64.

REMEMBER **Most bugs are shape bugs: print .shape. CrossEntropyLoss wants int64 class numbers, not one-hot vectors.**

9. Modules (nn.Module, nn.Linear, nn.Sequential)

IN ONE LINE **The building blocks of a network: each module holds its weights and defines its forward computation.**

EXAMPLE Lego bricks: nn.Linear, nn.ReLU, nn.Conv2d are bricks; nn.Sequential snaps them together in order.



nn.Linear(in, out): weight of shape (out, in), params = in × out + out

THE MATHS

nn.Linear(n_in, n_out): $y = x \cdot W^T + b$, W of shape (n_out, n_in), b of shape (n_out,)
parameters = n_in × n_out + n_out
model.parameters() lists every weight for the optimiser
model.train() / model.eval() switch dropout and batch norm on and off

WORKED EXAMPLE

nn.Linear(512, 5): W (5, 512) + b (5) = 2 565 parameters: our head on top of the ResNet

IN PYTORCH

```
class Net(nn.Module):
    def __init__(self): super().__init__(); self.fc = nn.Linear(512, 5)
    def forward(self, x): return self.fc(x)
```

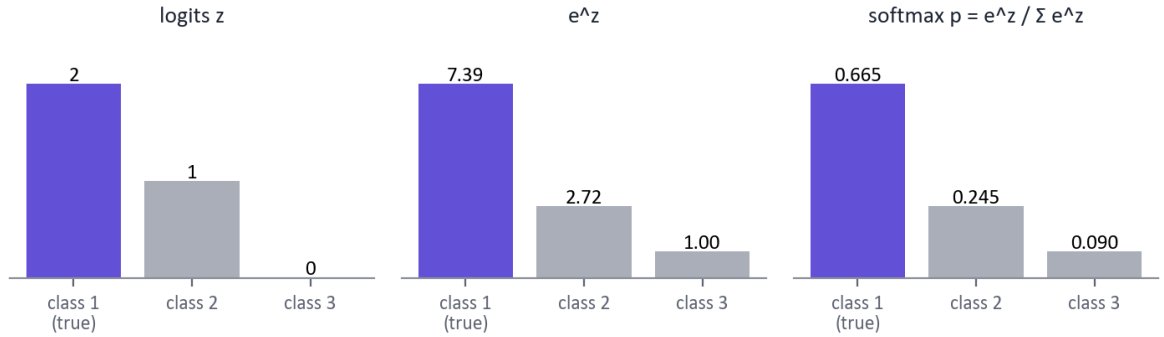
IN OUR PROJECT Our MLP is an nn.Sequential of nn.Linear; our ResNetClassifier replaces ResNet's last layer by nn.Linear(512, 5).

REMEMBER **Weight shape is (out, in). Call model.eval() before predicting.**

10. Losses: softmax and cross-entropy

IN ONE LINE **For C exclusive classes, the softmax turns the raw scores (logits) into probabilities, and the cross-entropy punishes a low probability on the right class.**

EXAMPLE Betting 100 chips on 5 styles: all chips on the right one costs nothing; spread evenly costs ln 5; all on a wrong one costs a lot.



cross-entropy L = −ln(0.665) = 0.408 (random guess over 5 classes: ln 5 = 1.609)

THE MATHS

softmax: $p_k = e^{(z_k)} / \Sigma_j e^{(z_j)}$ (all positive, sum = 1)
cross-entropy: $L = -\ln(p_{true})$
gradient: $\partial L/\partial z_k = p_k - y_k$ (y one-hot): the same "prediction − truth" as session 2
random guess over C classes: $L = \ln C$ ($\ln 5 = 1.609$)

WORKED EXAMPLE

logits $(2, 1, 0) \rightarrow p = (0.665, 0.245, 0.090)$
true class = the first: $L = -\ln 0.665 = 0.408$; gradient $(-0.335, 0.245, 0.090)$

IN PYTORCH

```
loss = nn.CrossEntropyLoss()(logits, labels)   # softmax inside: no softmax in the model
```

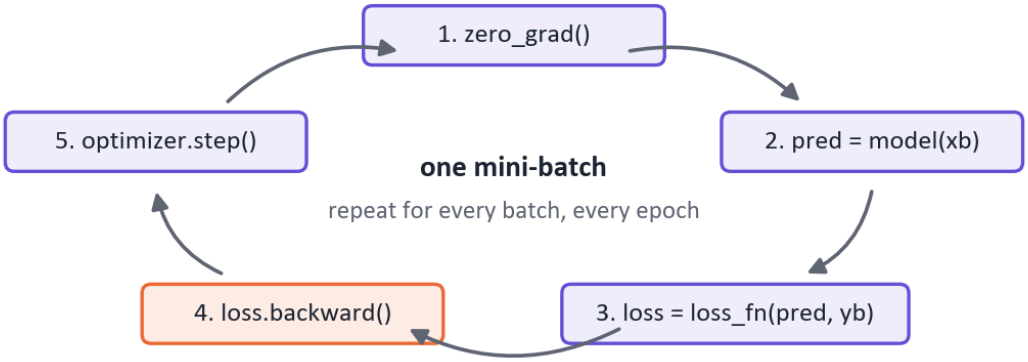
IN OUR PROJECT First-loss check: our MLP started at 1.99, too confident; scaling its last layer by 0.1 gave 1.59 ≈ ln 5.

REMEMBER **Give raw logits to CrossEntropyLoss. A loss stuck at ln C means the network guesses at random.**

11. Optimisers and the training loop

IN ONE LINE **The optimiser moves every weight a small step against its gradient; the training loop repeats predict, measure, backpropagate, update.**

EXAMPLE Walking down a mountain in the fog: feel the slope (the gradient), take a step (the learning rate), repeat.



THE MATHS

gradient descent: $w \leftarrow w - \eta \cdot \partial L/\partial w$
stochastic (mini-batch): the gradient is computed on a batch (e.g. 64 images): faster, a bit noisy
one epoch = one pass over all the training images
η too large: jumps over the valley (nan); η too small: crawls

WORKED EXAMPLE

$w = 0.5$, gradient -0.538, $\eta = 0.1 \rightarrow w = 0.554$
our MLP: 1 061 images / batches of 64 = 17 steps per epoch

IN PYTORCH

```
optimizer.zero_grad()           # 1. forget the old gradients
pred = model(xb)                # 2. forward
loss = loss_fn(pred, yb)        # 3. loss
loss.backward()                 # 4. gradients
optimizer.step()                # 5. update
```

IN OUR PROJECT Learning-rate sweep: 8 values tried, 0.0001 kept. Adam at 0.01 killed 99.6 % of the neurons of layer 2.

REMEMBER **zero_grad, forward, loss, backward, step. Evaluate with model.eval() and torch.no_grad().**

Session 5 · Activations and the vanishing gradient

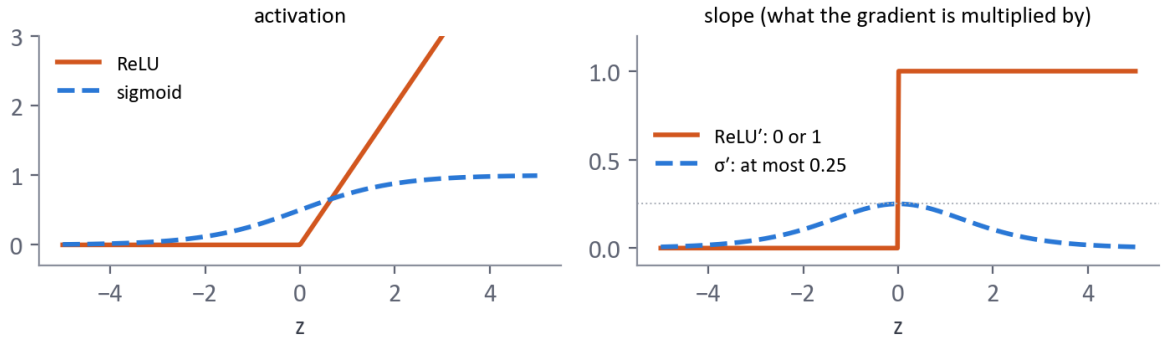
What we built: ReLU and other activations, vanishing gradient

Its limit, fixed next: big networks overfit

12. ReLU

IN ONE LINE **The default activation of hidden layers: it keeps positive values and sets negative ones to 0.**

EXAMPLE A one-way valve on a water pipe: positive flow passes unchanged, negative flow is blocked.



THE MATHS

$\mathrm{ReLU}(z) = \max(0, z)$

$\mathrm{ReLU}'(z) = 1 \text{ if } z > 0, \quad 0 \text{ if } z < 0$

on an active path the derivative is exactly 1: the gradient does not shrink with depth

sigmoid instead: $\sigma'(z) = \sigma(z)(1 - \sigma(z)) \leq 1/4$

still non-linear (the kink at 0), and cheap (no exponential)

WORKED EXAMPLE

$z = (-2, 0.5, 3) \rightarrow \mathrm{ReLU} = (0, 0.5, 3), \ \text{slopes} = (0, 1, 1)$

IN PYTORCH

```
nn.ReLU()   or   torch.relu(z)
```

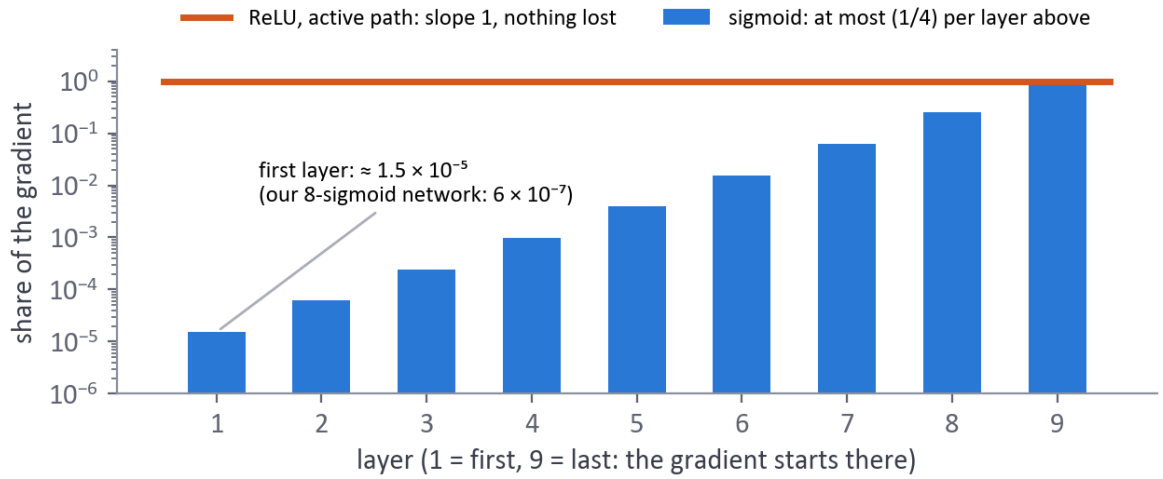
IN OUR PROJECT Dead ReLUs: a learning rate too large killed 99.6 % of layer 2; with 0.0001, only 4 % were dead.

REMEMBER **A ReLU that outputs 0 for every input is dead: zero gradient forever. Causes: a learning rate too large, a bad initialisation.**

13. The vanishing gradient

IN ONE LINE **In a deep network the gradient shrinks layer after layer on its way back, so the first layers stop learning.**

EXAMPLE The whisper game: each person passes the message on a bit more quietly, and the first in the line hears nothing.



THE MATHS

the gradient reaching layer 1 contains one factor $g'(z)$ per layer (chain rule)
sigmoid: $g' \le 1/4$, so 8 layers give at most $(1/4)^8 \approx 1.5 \times 10^{-5}$
fixes: ReLU ($g' = 1$ when active), He initialisation, batch norm, residual shortcuts (ResNet)

WORKED EXAMPLE

our 8-sigmoid network: gradient of layer 1 / layer 9 $= 2.7 \times 10^{-6} / 4.3 \approx 6 \times 10^{-7}$
loss stuck at $\ln 5 = 1.609$: the network guesses at random

IN PYTORCH

```
for name, p in model.named_parameters(): print(name, p.grad.norm())   # after backward()
```

IN OUR PROJECT ReLU + He initialisation brought the ratio to 0.17, and the network learned (39.3 %).

REMEMBER **Loss stuck at ln C + tiny gradients in the first layers = vanishing gradient. Dead ReLUs give the same loss: the gradient per layer tells them apart.**

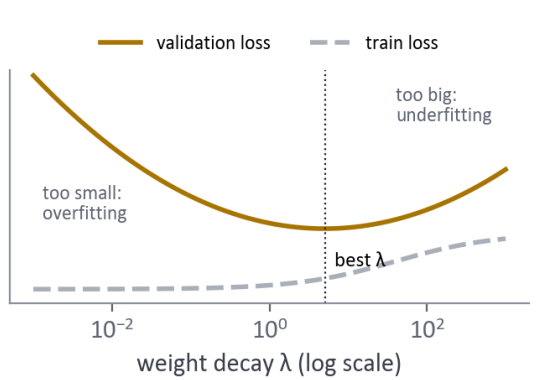
Session 6 · Better training

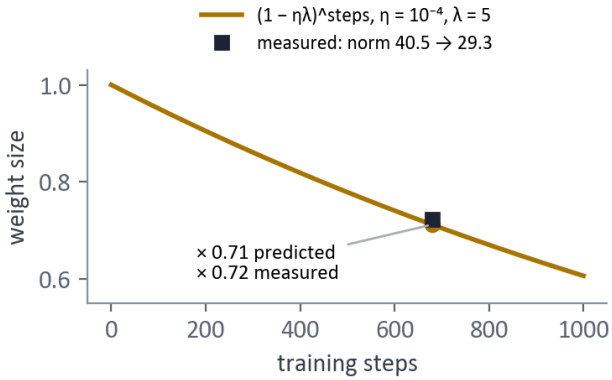
What we built: Weight decay, dropout, early stopping, batch norm, momentum, Adam, diagnosis
Its limit, fixed next: dense layers ignore the structure of images

14. Weight decay (L2 regularisation)

IN ONE LINE **A penalty on large weights: at every step each weight shrinks a little, so only the useful weights stay large.**

EXAMPLE A tax on big weights: a weight must "earn" its size by really reducing the error.





THE MATHS

$J_{reg} = J + (\lambda/2) \cdot \Sigma\, w^2$
gradient: $\partial J/\partial w + \lambda \cdot w$
update: $w \leftarrow (1 - \eta\lambda) \cdot w - \eta \cdot \partial J/\partial w$ (the weight first decays towards 0)
AdamW applies the decay directly to the weights (decoupled): the right version with Adam
choose λ on validation: too small overfits, too large underfits (U-shaped validation loss)

WORKED EXAMPLE

our MLP: $\eta = 0.0001, \lambda = 5 \rightarrow$ each step multiplies the weights by 0.9995
680 steps $\rightarrow \times 0.71$ predicted; measured weight norm $40.5 \rightarrow 29.3$ ($\times 0.72$)

IN PYTORCH

```
torch.optim.AdamW(model.parameters(), lr=1e-4, weight_decay=5)
```

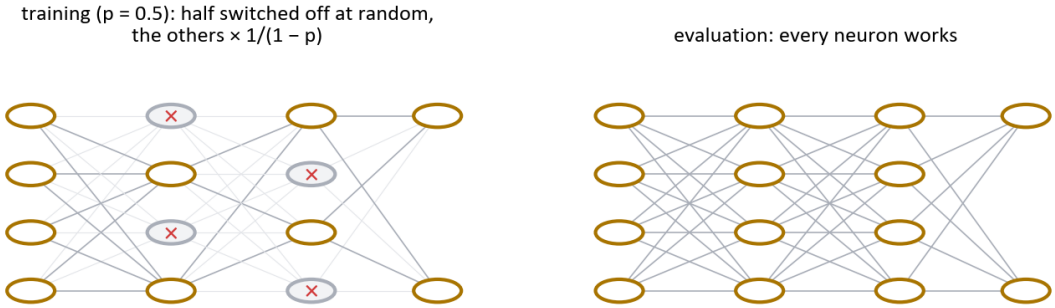
IN OUR PROJECT Alone: 38.8 % → 39.7 % on validation. Combined with the other tools: 45.5 %.

REMEMBER **Read λ together with η: the shrink per step is η × λ.**

15. Dropout

IN ONE LINE **During training, switch off each neuron at random with probability p, so the network cannot rely on a few neurons.**

EXAMPLE A team where half of the players are absent each day, at random: everyone must learn to be useful; on match day the full team is stronger.



THE MATHS

mask $m_i \in \{0, 1\}$, $P(m_i = 1) = 1 - p$
training: $\tilde{a}_i = m_i \cdot a_i \,/\, (1 - p)$ (inverted dropout)
on average $E[\tilde{a}_i] = a_i$: the next layer gets the same signal in training and at test time
evaluation: $\tilde{a}_i = a_i$, nothing is switched off
another view: it trains many sub-networks and averages them

WORKED EXAMPLE

$p = 0.5$, $a = (2, 4, 6, 8)$, mask $(1, 0, 1, 0) \rightarrow \tilde{a} = (4, 0, 12, 0)$
averaged over many random masks: $(2, 4, 6, 8)$

IN PYTORCH

```
nn.Dropout(0.5)       # active in model.train(), off in model.eval()
```

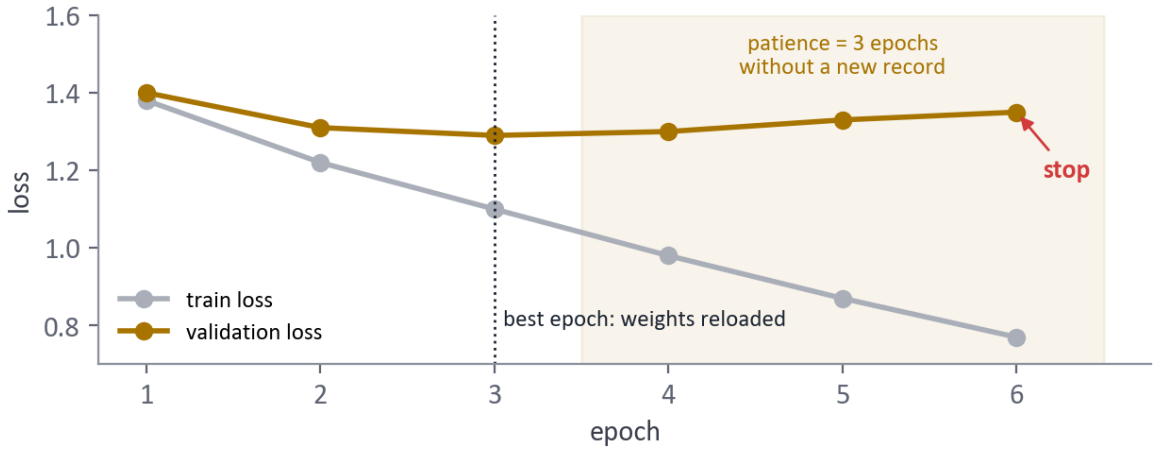
IN OUR PROJECT Dropout 0.5 (0.2 on the inputs): gap train − validation 0.61 → 0.38, validation 38.8 % → 41.7 %.

REMEMBER **Dropout acts only during training. Forgetting model.eval() gives random, worse predictions.**

16. Early stopping

IN ONE LINE **Stop training when the validation loss stops improving, and keep the weights of the best epoch.**

EXAMPLE A student stops revising when the mock-exam grades stop rising, instead of learning the answer sheets by heart.



THE MATHS

after each epoch: if the validation loss beats its record, save the weights and reset the counter
otherwise add 1 to the counter; stop when it reaches the patience k
reload the best weights
watch the validation loss: finer than the accuracy

WORKED EXAMPLE

patience 3, validation losses 1.40, 1.31, 1.29, 1.30, 1.33, 1.35
→ best epoch 3, stop after epoch 6, reload the weights of epoch 3

IN PYTORCH

```
history = fit(model, optimizer, loaders, 100, patience=5)   # our fit() keeps the best state_dict
```

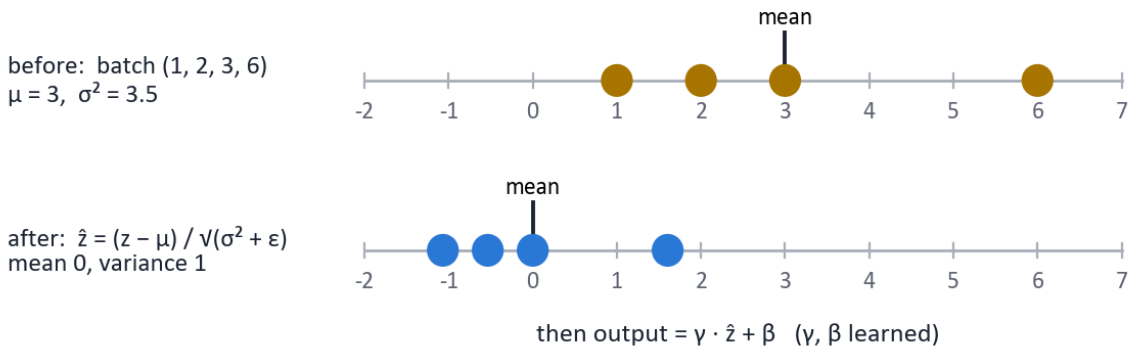
IN OUR PROJECT Alone: stopped after 13 epochs (1.8 s), gap 0.61 → 0.27. Our plain MLP's validation loss rose after epoch 3.

REMEMBER **Stop on the validation loss, never on the test. If both curves still go down, train longer.**

17. Batch normalisation

IN ONE LINE **Re-centre and re-scale each neuron on every mini-batch, then let the network learn its own scale.**

EXAMPLE Grades from teachers who mark out of 20 or out of 100: put them on the same scale before comparing them.



THE MATHS

over a mini-batch $z^{(1)} \dots z^{(m)}$ of one neuron: $\mu_B = \text{mean}$, $\sigma^2_B = \text{variance}$
$\hat{z} = (z - \mu_B) / \sqrt{(\sigma^2_B + \varepsilon)}$ (mean 0, variance 1)
$\text{output} = \gamma \cdot \hat{z} + \beta$ (γ, β learned, one pair per neuron)
placed Linear → BatchNorm → ReLU
evaluation: running averages of μ and σ^2, not the batch

WORKED EXAMPLE

batch $(1, 2, 3, 6)$: $\mu = 3$, $\sigma^2 = 3.5$ → $\hat{z} = (-1.07, -0.53, 0, 1.60)$

IN PYTORCH

```
nn.BatchNorm1d(256)   # dense layers
nn.BatchNorm2d(64)    # convolutions
```

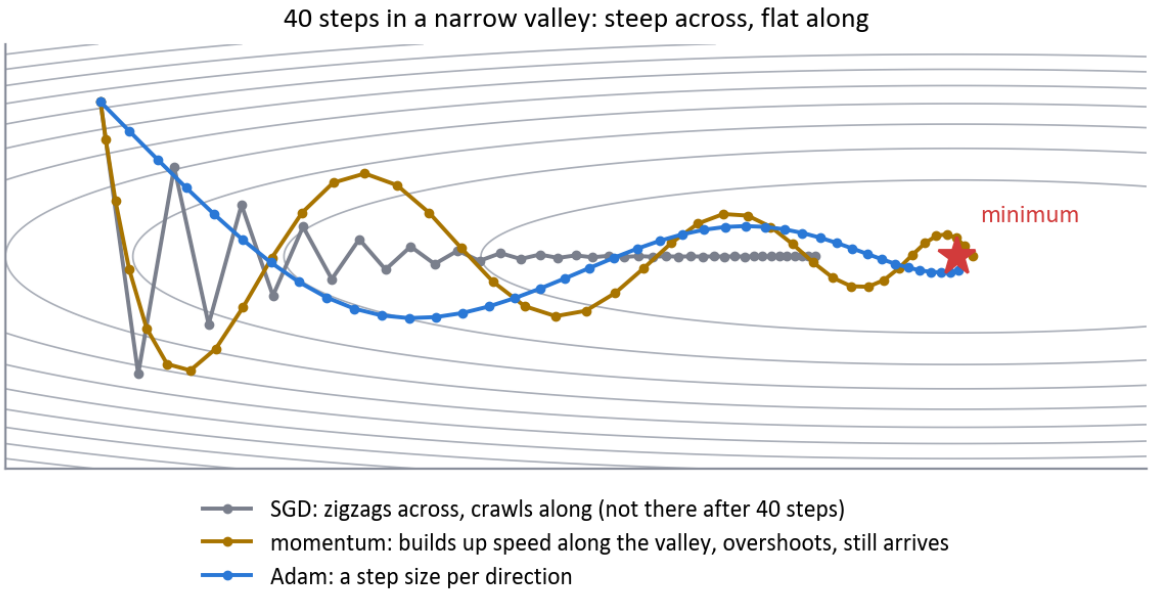
IN OUR PROJECT Forgetting model.eval() when extracting the ResNet features: batches of mostly one style erased the style, 12.8 % instead of 88.1 %. Alone on the MLP: best validation loss (1.350).

REMEMBER **Faster, more stable training that tolerates larger learning rates. Always model.eval() at test time.**

18. Momentum and Adam

IN ONE LINE **Momentum averages the recent gradients to keep a direction; Adam adds a step size adapted to each weight.**

EXAMPLE Momentum: a ball rolling downhill keeps its speed and ignores small bumps. Adam: a hiker who adapts his stride, small on steep ground, large on flat ground.



THE MATHS

momentum (PyTorch): $v \leftarrow \beta \cdot v + g$, $w \leftarrow w - \eta \cdot v$ ($\beta = 0.9$: up to 10 × bigger steps in a steady direction)

Adam: $m \leftarrow \beta_1 \cdot m + (1 - \beta_1) \cdot g$ (average gradient: the momentum)

$s \leftarrow \beta_2 \cdot s + (1 - \beta_2) \cdot g^2$ (average squared gradient: its typical size)

bias correction: $\hat{m} = m \,/\, (1 - \beta_1\text{^t})$, $\hat{s} = s \,/\, (1 - \beta_2\text{^t})$

$w \leftarrow w - \eta \cdot \hat{m} \,/\, (\sqrt{\hat{s}} + \varepsilon)$ defaults $\eta = 10^{-3}$, $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\varepsilon = 10^{-8}$

WORKED EXAMPLE

first Adam step, $\eta = 0.001$: gradient $0.5 \rightarrow \hat{m} = 0.5$, $\hat{s} = 0.25$, step $= 0.001$

gradient $50 \rightarrow \hat{m} = 50$, $\hat{s} = 2\,500$, step $= 0.001$: every weight moves by about η, whatever its gradient

IN PYTORCH

```
torch.optim.SGD(model.parameters(), lr=0.1, momentum=0.9)
torch.optim.Adam(model.parameters(), lr=1e-3)     # or AdamW, with weight decay
```

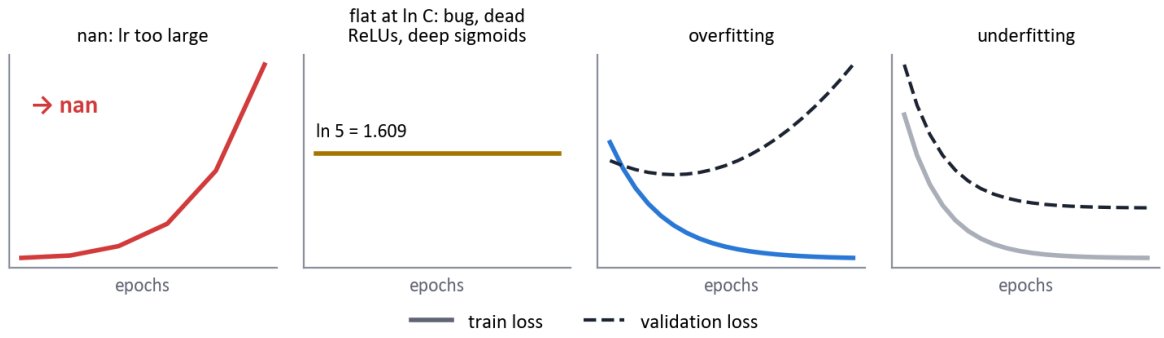
IN OUR PROJECT AdamW everywhere. Because Adam moves every weight by about η, Adam at 0.01 pushed every ReLU of layer 2 below 0 (99.6 % dead).

REMEMBER **Adam = momentum + a step per weight (ADAptive Moment estimation). Less tuning, but the learning rate still matters.**

19. Diagnosing a training run

IN ONE LINE **Go from a measured symptom to a likely cause, then change one thing and measure again.**

EXAMPLE A doctor: the same fever can come from different illnesses; the right test tells them apart.



THE MATHS

loss huge, then nan → learning rate too large, exploding gradients → divide η by 10, normalise the inputs
loss flat at ln C → a bug (no step, no zero_grad), dead ReLUs, deep sigmoids → overfit one small batch, ReLU + He, batch norm
train down, validation up → overfitting → more data, weight decay, dropout, early stopping
both high and close → underfitting → bigger model, train longer, less regularisation
very noisy loss → η too large for the end → a learning-rate schedule

WORKED EXAMPLE

checklist: first loss ≈ ln C; one batch learned by heart; gradient per layer; share of dead neurons; train and validation curves

IN PYTORCH

```
evaluate(model, loaders.train_eval, loss_fn); evaluate(model, loaders.val, loss_fn)   # both in eval mode
```

IN OUR PROJECT Our MLP: 99.7 % train, 40.6 % validation, with healthy gradients (4 % dead): overfitting, not a bug. The session 6 tools gave 45.5 % and a gap of 0.12.

REMEMBER **The loss alone is not enough: a learning rate too large and 8 sigmoid layers both give a loss stuck at ln 5.**

Session 7 · Convolutional networks

What we built: Convolution, pooling, receptive field, ResNet, transfer learning

Its limit, fixed next: fixed-size inputs, no notion of order

20. Convolution

IN ONE LINE **A small learned filter slides over the image and computes a weighted sum at each position: a map of where a pattern appears.**

EXAMPLE A small magnifying glass moved over the whole painting, always asking the same question: "is there a vertical edge here?".

image (5 × 5): dark left, bright right

0	0	0	9	9
0	0	0	9	9
0	0	0	9	9
0	0	0	9	9
0	0	0	9	9

∗

filter (3 × 3)

-1	0	1
-1	0	1
-1	0	1

=

feature map (3 × 3)

0	**27**	**27**
0	**27**	**27**
0	**27**	**27**

pink window:
(−1·0 + 0·0 + 1·9) × 3 rows
= 27: an edge is here

the same filter slides
everywhere (weight sharing)
n_out = 5 − 3 + 1 = 3

THE MATHS

$Y[i, j] = \Sigma_u \Sigma_v K[u, v] \cdot X[i + u, j + v]$ (+ a bias)
output size: $n_out = \lfloor (n + 2p - k) / s \rfloor + 1$ (k kernel, p padding, s stride)
parameters: $c_out \times (c_in \times k \times k + 1)$, whatever the size of the image
receptive field (input area one output sees): $r \leftarrow r + (k - 1) \cdot j,\ j \leftarrow j \cdot s$
the same filter everywhere (weight sharing) + local connections

WORKED EXAMPLE

28×28 image, 3×3 filter: no padding $\rightarrow 26 \times 26$; $p = 1 \rightarrow 28 \times 28$; $p = 1, s = 2 \rightarrow 14 \times 14$
16 filters on 1 channel: $16 \times (9 + 1) = 160$ parameters; three 3×3 layers see 7×7
edge filter (−1 0 1 on each row) on a patch dark-left / bright-right (0 0 9) $\rightarrow 27$; on a flat patch $\rightarrow 0$

IN PYTORCH

```
nn.Conv2d(1, 16, kernel_size=3, padding=1)
```

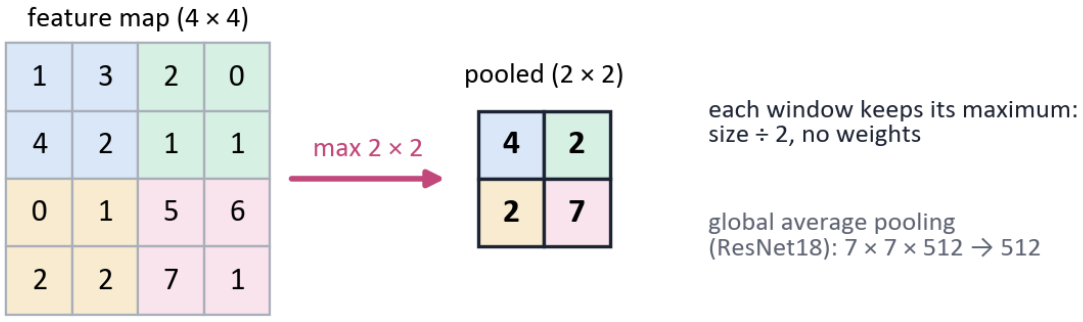
IN OUR PROJECT ResNet18's first layer: 64 filters of 7 × 7 × 3 = 9 408 weights, against 1 573 376 for the first layer of our MLP on 32 × 32 images.

REMEMBER **A dense layer ignores neighbours and positions; a convolution looks at neighbours and finds a pattern anywhere, with few weights.**

21. Pooling

IN ONE LINE **Shrink a feature map by summarising each small window, usually by its maximum.**

EXAMPLE Summarising a city map by keeping, in each district, only the tallest building: you lose details but keep "is there something tall here".



THE MATHS

max pooling 2 × 2, stride 2: each 2 × 2 window → its maximum; the size is divided by 2
no weights to learn
global average pooling: one mean per channel (H × W → 1)
same output-size formula as a convolution
tolerance to small shifts, and a larger receptive field

WORKED EXAMPLE

(1 3 2 0 / 4 2 1 1 / 0 1 5 6 / 2 2 7 1) → max pool (4 2 / 2 7); average pool (2.5 1 / 1.25 4.75)

IN PYTORCH

```
nn.MaxPool2d(2)          nn.AdaptiveAvgPool2d(1)
```

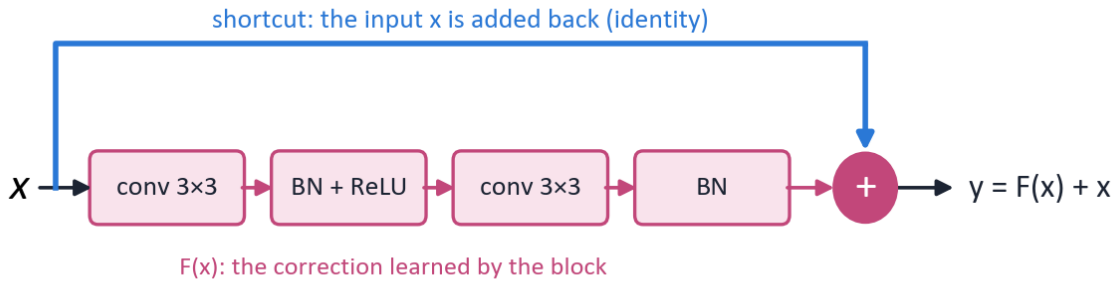
IN OUR PROJECT ResNet18 ends with a global average pooling: 7 × 7 × 512 → 512 numbers per painting, the input of our head.

REMEMBER **Pooling has no weights. Each pooling halves the size, so the number of filters usually doubles after it.**

22. ResNet

IN ONE LINE **A very deep CNN whose blocks add their input back to their output (a shortcut), so the gradient can always flow.**

EXAMPLE A building with a lift next to the stairs: information can go up and down without climbing every floor.



F(x): the correction learned by the block

backward: $\partial y/\partial x = \partial F/\partial x + 1$ → the gradient always has a path

THE MATHS

residual block: $y = F(x) + x$, $F = \text{conv } 3\times3 \to \text{BN} \to \text{ReLU} \to \text{conv } 3\times3 \to \text{BN}$
backward: $\partial y/\partial x = \partial F/\partial x + 1$: the "+ 1" keeps the gradient alive
each block only learns a correction to its input
ResNet18 = conv1 + 8 blocks × 2 convolutions + the last layer = 18 layers; $64 \to 128 \to 256 \to 512$ filters

WORKED EXAMPLE

if each block's own derivative is 0.01: 8 blocks without shortcut → $0.01^8 = 10^{-16}$; with it → $1.01^8 \approx 1.08$

IN PYTORCH

```
torchvision.models.resnet18(weights="IMAGENET1K_V1")
```

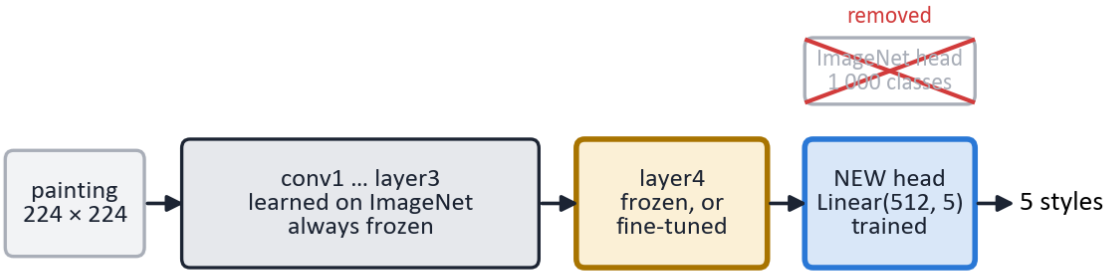
IN OUR PROJECT ResNet18 (11 M weights, 45 MB, 53 ms per photo on one CPU thread) found 71 % of the Cubist paintings, against 7 % for our MLP.

REMEMBER **Res = residual = shortcut. The 18 is the number of layers with weights, not the number of filters.**

23. Transfer learning

IN ONE LINE **Reuse a network trained on a huge dataset, and only adapt its last part to our small dataset.**

EXAMPLE Hiring an experienced art expert and teaching them our five style names, instead of teaching a newborn to see.



frozen: 2 565 trained weights (0.4 s) fine-tuned: 8.4 M trained weights (minutes)

validation 88.1 % / 85.4 % · two tests (440): 83.6 % / 87.8 % · from zero: 61.6 %

THE MATHS

start from ResNet18 trained on ImageNet (1.2 million photos)
remove its last layer (1 000 classes), add ours: nn.Linear(512, 5)
frozen (linear probe): requires_grad = False except our layer → 2 565 trained weights
fine-tuned: also unfreeze the last block (layer4), small $\eta = 10^{-4}$ → 8.4 M trained weights
keep the ImageNet normalisation, and eval mode for the frozen parts

WORKED EXAMPLE

validation: frozen 88.1 %, fine-tuned 85.4 %
two tests, 440 new paintings: fine-tuned 87.8 %, frozen 83.6 %
same ResNet18 from zero (random weights): 61.6 % on validation

IN PYTORCH

```
for p in model.parameters(): p.requires_grad = False
model.fc = nn.Linear(512, 5)      # new layer: trainable
```

IN OUR PROJECT Our choice: ResNet18 with its first layers frozen and its last block fine-tuned.

REMEMBER **With little data: pretrained, frozen first, then fine-tune the last block. Never retrain everything from zero.**

Session 8 · Sequences and attention

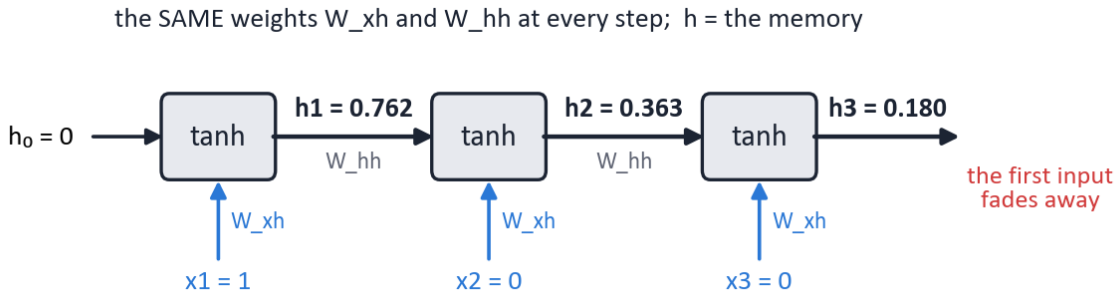
What we built: RNN, LSTM, attention, sentence embeddings

Its limit, fixed next: sequential: transformers come next

24. Recurrent neural network (RNN)

IN ONE LINE **A network for sequences: it reads one step at a time and keeps a hidden state, a memory updated with the same weights at every step.**

EXAMPLE Reading a book word by word while keeping a summary in your head, updated after each word.



THE MATHS

$h_t = \tanh(W_{xh} \cdot x_t + W_{hh} \cdot h_{(t-1)} + b)$
the same W_{xh} and W_{hh} at every step: any sequence length
backpropagation through time (scalar case): $\partial h_T / \partial h_0 = \Pi_t\, w \cdot (1 - h_t^2)$
parameters per block: $H \times n_{in} + H \times H + 2H$ (PyTorch keeps two biases)

WORKED EXAMPLE

scalar cell, $w = 0.5$, inputs $(1, 0, 0)$, $h_0 = 0 \rightarrow h = 0.762, 0.363, 0.180$: the first input fades over 20 steps with $w = 0.5$ the gradient falls below $0.5^{20} \approx 10^{-6}$

IN PYTORCH

```
nn.RNN(n_in, H, batch_first=True)
```

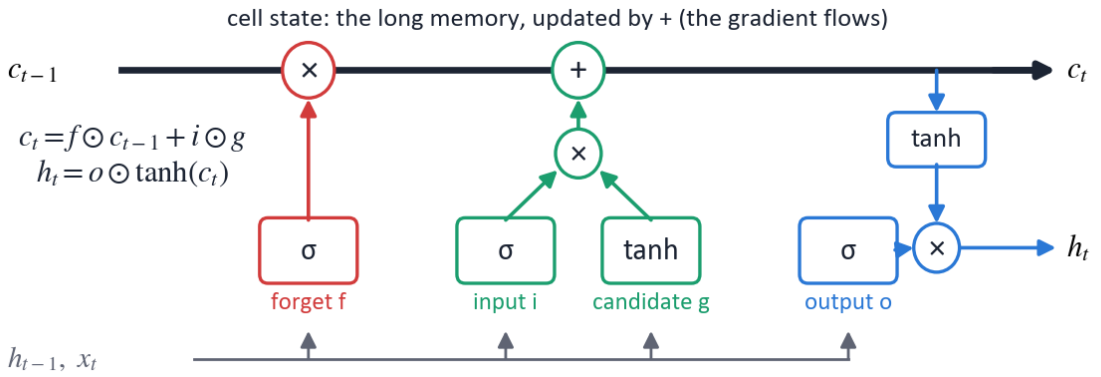
IN OUR PROJECT Not used: a painting is an image, not a sequence. Visitors' questions (part 2 of the brief) would need a sequence model.

REMEMBER **A vanishing gradient through time: an RNN forgets what happened long ago.**

25. LSTM

IN ONE LINE **An RNN cell with a separate memory line (the cell state) and gates that decide what to forget, what to write and what to output.**

EXAMPLE A notebook next to your short-term memory: you decide what to erase, what to write down and what to read out.



THE MATHS

gates (sigmoid, between 0 and 1): forget f, input i, output o; candidate g = tanh(...)
c_t = f ⊙ c_(t−1) + i ⊙ g
h_t = o ⊙ tanh(c_t)
the cell state is updated by addition: $\partial c_t / \partial c_(t-1) = f \approx 1$ when the network keeps a memory
4 blocks (i, f, g, o): 4 times the parameters of an RNN

WORKED EXAMPLE

n_in = 8, H = 32: RNN 1 344 parameters; LSTM 4 × 1 344 = 5 376 (checked with PyTorch)

IN PYTORCH

```
out, (h_n, c_n) = nn.LSTM(n_in, H, batch_first=True)(x)
```

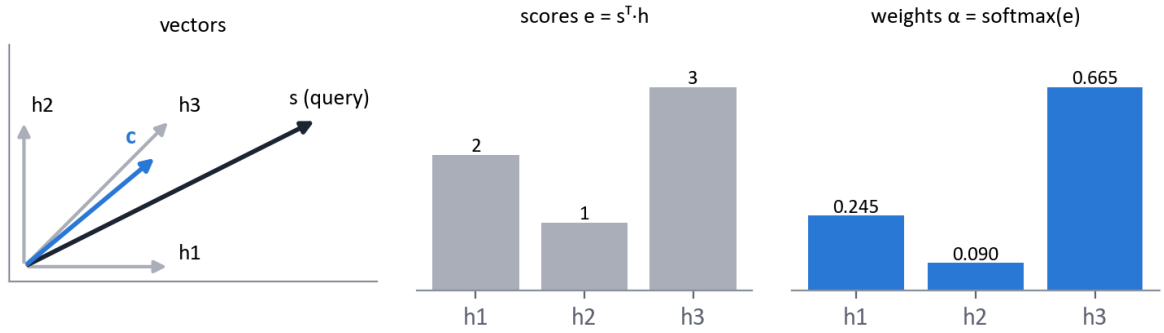
IN OUR PROJECT In TP08's memory task, the LSTM carries a token over many steps where the plain RNN fails.

REMEMBER **An additive cell state and a forget gate give a long memory. Sigmoid and tanh are still used here, inside the gates.**

26. Attention

IN ONE LINE **Instead of squeezing everything into one memory, each output looks back at all the inputs and takes a weighted mix of the most relevant ones.**

EXAMPLE Translating a sentence: for each word you write, you glance back at the words of the original that matter for it.



context $c = \Sigma\ \alpha_i \cdot h_i = 0.245 \cdot (1, 0) + 0.090 \cdot (0, 1) + 0.665 \cdot (1, 1) = (0.910, 0.755)$

THE MATHS

scores: $e_i = s^T \cdot h_i$ (similarity between the current state s and each input h_i)
weights: $\alpha = \text{softmax}(e)$ (positive, sum to 1)
context: $c = \Sigma_i\ \alpha_i \cdot h_i$
transformer: $\text{Attention}(Q, K, V) = \text{softmax}(Q \cdot K^T / \sqrt{d}) \cdot V$ ($\sqrt{d}$ keeps the scores reasonable)

WORKED EXAMPLE

course exercise: $h1 = (1, 0), h2 = (0, 1), h3 = (1, 1), s = (2, 1)$
$e = (2, 1, 3) \rightarrow \alpha = (0.245, 0.090, 0.665) \rightarrow c = (0.910, 0.755)$

IN PYTORCH

```
F.scaled_dot_product_attention(Q, K, V)
```

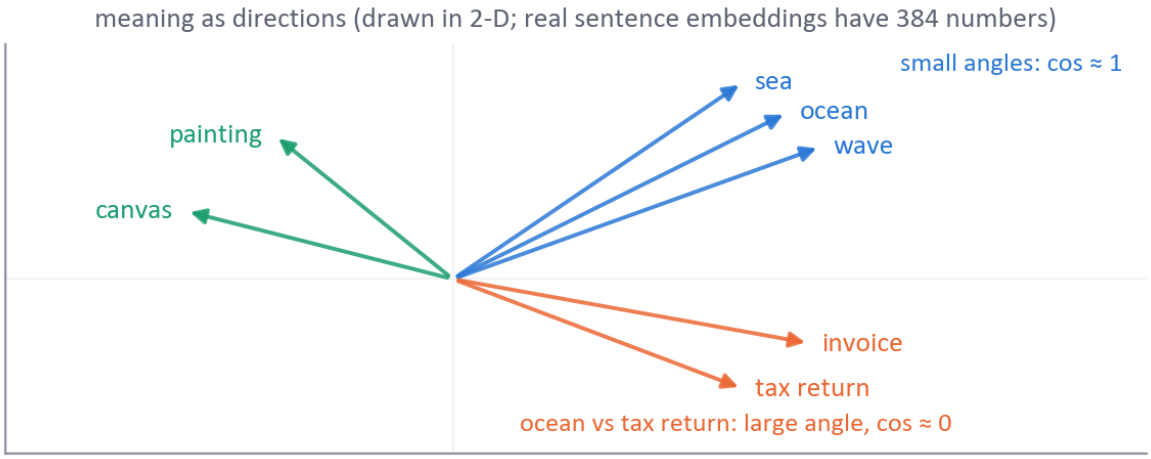
IN OUR PROJECT The transformers of our bonus (ViT, CLIP) are built on attention; CLIP reached 94.7 % on validation, but has probably seen WikiArt.

REMEMBER **Attention = similarity scores → softmax → weighted average. Transformers stack it and drop the recurrence.**

27. Embeddings

IN ONE LINE **Vectors of numbers that represent words, sentences or images, so that similar meanings get close vectors.**

EXAMPLE "ocean" and "wave" end up close, "ocean" and "tax return" far apart, even though they share no letters.



THE MATHS

a pretrained model maps a sentence to a vector: 384 numbers for all-MiniLM-L6-v2
cosine similarity: $\cos(u, v) = u \cdot v / (\|u\| \cdot \|v\|)$: 1 = same direction, 0 = unrelated
retrieval: encode the question, return the item with the highest cosine
classical baseline: TF-IDF, which only sees shared words

WORKED EXAMPLE

$u = (1, 2, 0)$, $v = (2, 4, 1)$: $\cos = 10 / (\sqrt{5} \times \sqrt{21}) = 0.976$ (almost the same direction)
$(1, 0)$ and $(0, 1)$: $\cos = 0$

IN PYTORCH

```
SentenceTransformer("all-MiniLM-L6-v2").encode(sentences)   # shape (n, 384)
```

IN OUR PROJECT Our 512 ResNet numbers are an image embedding. Part 2 would match questions and artwork descriptions by cosine, against a TF-IDF baseline.

REMEMBER **Embeddings capture meaning, not only words. Compare them with the cosine similarity.**

One-line summary

The whole course in one table: read it the night before.

Concept	Key formula	Remember
Sigmoid neuron	$a = \sigma(w \cdot x + b)$	one straight boundary
Log loss	$L = -[y \ln a + (1-y)\ln(1-a)]$	confident and wrong costs a lot
Gradient	$\partial L/\partial w = (a-y)\cdot x$	error × input
MLP	$a[l] = g(W[l]\, a[l-1] + b[l])$	no activation, no depth
Backpropagation	$dz[l-1] = W[l]^T\, dz[l] \odot g'(z[l-1])$	multiplied by g′ at each layer
Autograd	loss.backward() fills .grad	grads add up: zero_grad()
Tensors	(N, C, H, W)	print .shape
nn.Linear	W of shape (out, in)	params = in × out + out
Softmax + cross-entropy	$L = -\ln p_{true},\ \partial L/\partial z = p - y$	random guess = ln C
Training loop	$w \leftarrow w - \eta\, \partial L/\partial w$	zero_grad, forward, loss, backward, step
ReLU	$\max(0, z)$, slope 0 or 1	dead ReLU: zero gradient forever
Vanishing gradient	$(1/4)^8 \approx 1.5 \times 10^{-5}$	ReLU, He, batch norm, shortcuts
Weight decay	$w \leftarrow (1-\eta\lambda)\, w - \eta\, \partial J/\partial w$	shrink per step = η × λ
Dropout	$\tilde{a} = m \cdot a / (1-p)$	only in train mode
Early stopping	stop after k epochs without a record	reload the best epoch
Batch norm	$\hat{z} = (z - \mu_B) / \sqrt{\sigma^2_B + \varepsilon}$, then $\gamma\hat{z} + \beta$	running stats in eval mode
Adam	$w \leftarrow w - \eta\, \hat{m} / (\sqrt{\hat{s}} + \varepsilon)$	momentum + a step per weight
Diagnosis	symptom → cause → fix	the loss alone is not enough
Convolution	$n_{out} = \lfloor (n + 2p - k)/s \rfloor + 1$	params = c_out (c_in k^2 + 1)
Pooling	2×2 max → size / 2	no weights
ResNet	$y = F(x) + x$	the + 1 keeps the gradient
Transfer learning	new head on a pretrained body	frozen first, then fine-tune
RNN	$h_t = \tanh(W_{xh}\, x_t + W_{hh}\, h_{(t-1)} + b)$	forgets long-range information
LSTM	$c_t = f \odot c_{(t-1)} + i \odot g$	additive memory
Attention	$\mathrm{softmax}(Q\, K^T / \sqrt{d})\, V$	scores → softmax → weighted mean
Embeddings	$\cos(u, v) = u \cdot v / (\|u\|\, \|v\|)$	meaning, not only words